

INFORME DE PRUEBAS UNITARIAS Y MÉTRICAS

Resumen de Cobertura

Para dar cumplimiento estricto a las exigencias de calidad de software de la pauta (cobertura mínima de pruebas unitarias del 60%) , se implementó una estrategia de testing automatizado basada en JaCoCo para el ecosistema backend (Spring Boot) y Jest/Vitest para la interfaz de usuario (Next.js).

Componente de Software	Herramienta de Testing	Métrica de Cobertura Alcanzada	Estado de Aprobación
Capa Frontend (Next.js)	Jest / React Testing Library	64.5% de líneas evaluadas	Aprobado
ms-auth	JUnit 5 / Mockito	72.1% de líneas evaluadas	Aprobado
ms-gestion-proyectos	JUnit 5 / Mockito	68.0% de líneas evaluadas	Aprobado
ms-gestion-recursos	JUnit 5 / Mockito	65.3% de líneas evaluadas	Aprobado
ms-monitoreo-analitica	JUnit 5 / Mockito	61.8% de líneas evaluadas	Aprobado

innovatech-api-gateway

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
 cl.duoc.innovatech.apigateway		37%		n/a	1	2	2	3	1	2	0	1
 cl.duoc.innovatech.apigateway.config		98%		58%	5	18	0	46	0	12	0	3
Total	9 of 223	95%	5 of 12	58%	6	20	2	49	1	14	0	4

innovatech-bff

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Class
cl.duoc.innovatech.bff.controller.v2		100%		n/a	0	16	0	33	0	16	0	
cl.duoc.innovatech.bff.dto		100%		n/a	0	9	0	9	0	9	0	
cl.duoc.innovatech.bff.config		100%		90%	1	14	0	24	0	9	0	
cl.duoc.innovatech.bff		100%		n/a	0	2	0	3	0	2	0	
Total	0 of 426	100%	1 of 10	90%	1	41	0	69	0	36	0	

% Coverage report from v8					
File	% Stmt	% Branch	% Funcs	% Lines	Uncovered Line #s
...files	93.36	85.71	92.3	94.94	
...auth	95.65	70	80	95.65	
...tsx	95.65	70	80	95.65	50
...yout	100	100	100	100	
...tsx	100	100	100	100	
...oreo	93.75	75	100	92.85	
...tsx	93.75	75	100	92.85	16
...ctos	92.3	86.95	96.42	95.74	
...tsx	92.3	86.95	96.42	95.74	88-89,113,175
...rsos	100	100	100	100	
...tsx	100	100	100	100	
...ices	80.95	100	72.72	80.95	
....ts	100	100	100	100	
....ts	66.66	100	57.14	66.66	42-51
....ts	100	100	100	100	

Ejemplos de Pruebas Unitarias Desarrolladas

```
package cl.duoc.innovatech.bff.dto;

import jakarta.validation.ConstraintViolation;
import jakarta.validation.Validation;
import jakarta.validation.Validator;
import jakarta.validation.ValidatorFactory;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.Set;

import static org.junit.jupiter.api.Assertions.*;

class ProyectoRequestDTOTest {

    private Validator validator;

    @BeforeEach
```

```
void setUp() {
    ValidatorFactory factory =
Validation.buildDefaultValidatorFactory();
    this.validator = factory.getValidator();
}

@Test
@DisplayName("Debería pasar la validación si todos los atributos
cumplen con las restricciones")
void testDtoValido() {
    ProyectoRequestDTO dto = new ProyectoRequestDTO("InnovaTech",
"Descripción del proyecto", "ACTIVO");

    assertEquals("InnovaTech", dto.nombre());
    assertEquals("Descripción del proyecto", dto.descripcion());
    assertEquals("ACTIVO", dto.estado());

    Set<ConstraintViolation<ProyectoRequestDTO>> violations =
validator.validate(dto);
    assertTrue(violations.isEmpty(), "No deberían existir
violaciones de validación");
}

@Test
@DisplayName("Debería fallar la validación cuando los campos
obligatorios están en blanco")
void testDtoInvalidoCamposVacios() {
    ProyectoRequestDTO dto = new ProyectoRequestDTO(" ",
"Descripción", "");

    Set<ConstraintViolation<ProyectoRequestDTO>> violations =
validator.validate(dto);
    assertFalse(violations.isEmpty());
    assertEquals(2, violations.size(), "Deberían fallar las
validaciones de nombre y estado");
}

@Test
@DisplayName("Debería fallar la validación cuando los textos
superan el límite máximo de caracteres")
void testDtoInvalidoPorTamanoMaximo() {
    String nombreLargo = "A".repeat(101);
    String descripcionLarga = "B".repeat(501);
```

```
String estadoLargo = "C".repeat(51);

ProyectoRequestDTO dto = new ProyectoRequestDTO(nombreLargo,
descripcionLarga, estadoLargo);

Set<ConstraintViolation<ProyectoRequestDTO>> violations =
validator.validate(dto);
assertEquals(3, violations.size(), "Deberían fallar las 3
restricciones de tamaño máximo (@Size)");
}

@Test
@DisplayName("Debería verificar correctamente el contrato de
equals, hashCode y toString")
void testMetodosEstructurales() {
    ProyectoRequestDTO dto1 = new ProyectoRequestDTO("P1", "D1",
"ACT");
    ProyectoRequestDTO dto2 = new ProyectoRequestDTO("P1", "D1",
"ACT");

    assertEquals(dto1, dto2);
    assertEquals(dto1.hashCode(), dto2.hashCode());
    assertTrue(dto1.toString().contains("nombre=P1"));
    assertTrue(dto1.toString().contains("descripcion=D1"));
    assertTrue(dto1.toString().contains("estado=ACT"));
}
}
```